DATA SCIENCE

Matplotlib & Seaborn

Master the Matplotlib figure/axes model and Seaborn's statistical plots to turn raw data into clear, publication-quality visuals and to choose the right chart every time.

matplotlib · seaborn · data-visualization · python · plotting · eda · charts

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. 1. What They Are and Why They Matter
2. 2. Setup and Getting Started
3. 3. The Figure / Axes Model — the One Thing to Understand
4. 4. Core Chart Types in Matplotlib
5. 5. Subplots and Layout
6. 6. Seaborn: Statistical Plots in One Line
7. 7. Worked Mini-Example: An EDA Panel
8. 8. Choosing the Right Chart
9. 9. Cheat-Sheet Reference
10. 10. Best Practices
11. 11. Common Pitfalls
12. 12. Interview Questions & Answers
13. 13. Further Learning

Matplotlib is the foundational plotting library for Python — low-level, explicit, and able to draw essentially anything. Seaborn sits on top of it, trading a little control for a lot of convenience: statistical charts, sensible defaults, and tidy-data awareness in one line. Learn Matplotlib's figure/axes model once and every chart in the ecosystem (Seaborn, pandas `.plot()`, scikit-learn displays) becomes customisable, because they all return Matplotlib objects.

1. What They Are and Why They Matter

A visualisation is a function from data to pixels whose job is to let a human reason about numbers faster than a table ever could. Matplotlib (first released 2003) is the *engine*: it owns the canvas, the axes, the ticks, and every line and glyph. Seaborn (first released 2014) is a *statistical grammar* layered on top — it reads tidy DataFrames, computes aggregations and confidence intervals for you, and emits attractive Matplotlib figures you can still tweak afterwards.

MATPLOTLIB

Total control, verbose. You place every element. The substrate for the whole Python viz stack.

SEABORN

Concise statistical charts from tidy data: distributions, relationships, categories, correlation — with themes that look good by default.

THEY COMPOSE

Seaborn draws onto a Matplotlib `Axes`. Start in Seaborn for speed, drop to Matplotlib for the last 10% of polish.

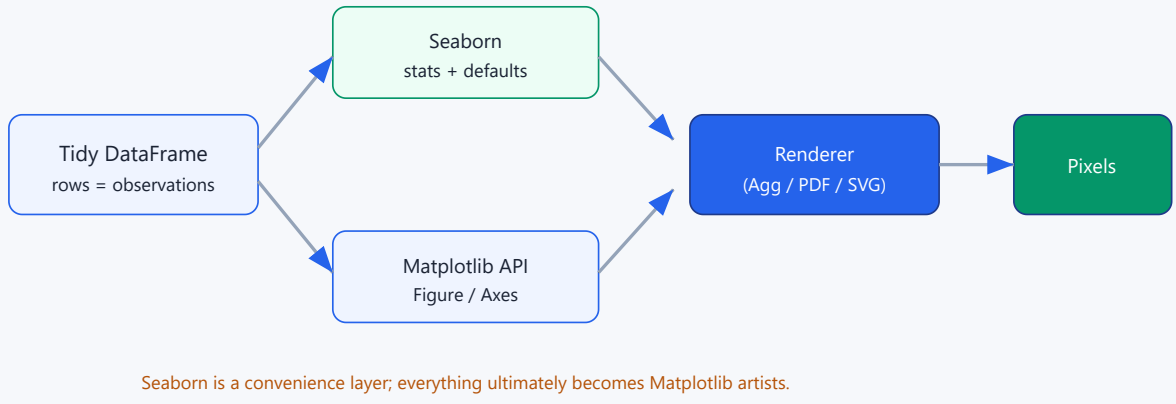


Fig 1 — Data flows through Seaborn (optional) into Matplotlib's Figure/Axes, then to a backend renderer.

Mental model

Think of a **Figure** as the sheet of paper and each **Axes** as one chart drawn on it. Almost every plotting call ultimately adds *artists* (lines, patches, text) to an Axes.

2. Setup and Getting Started

BASH

```bash
# Install into a virtual environment
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install matplotlib seaborn pandas numpy

# Versions this guide targets (2026-era)
python -c "import matplotlib, seaborn; print(matplotlib.__version__, seaborn.__version
# e.g. 3.9.x  0.13.x
```

The canonical imports. Note Seaborn is conventionally `sns` and Matplotlib's stateful interface is `plt`:

PYTHON

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# A good global baseline: Seaborn theme + readable DPI
sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.dpi"] = 110        # crisper inline figures
plt.rcParams["savefig.bbox"] = "tight"  # trim whitespace on export
```

✦ **Backends**

In Jupyter, figures render inline automatically. In a plain script you need `plt.show()` to open a window, or `fig.savefig("out.png")` to write a file. On a headless server set a non-interactive backend early: `import matplotlib; matplotlib.use("Agg")`.

3. The Figure / Axes Model — the One Thing to Understand

Matplotlib has **two interfaces**, and mixing them up is the single biggest source of confusion.

PYPLOT (IMPLICIT)

Commands like `plt.plot()` act on a hidden "current axes". Fine for quick, one-chart throwaways. State is global and surprising with multiple plots.

OBJECT-ORIENTED (EXPLICIT)

`fig, ax = plt.subplots()` then call methods on `ax`. Verbose but unambiguous. **Use this** for anything reusable, in functions, or with subplots.

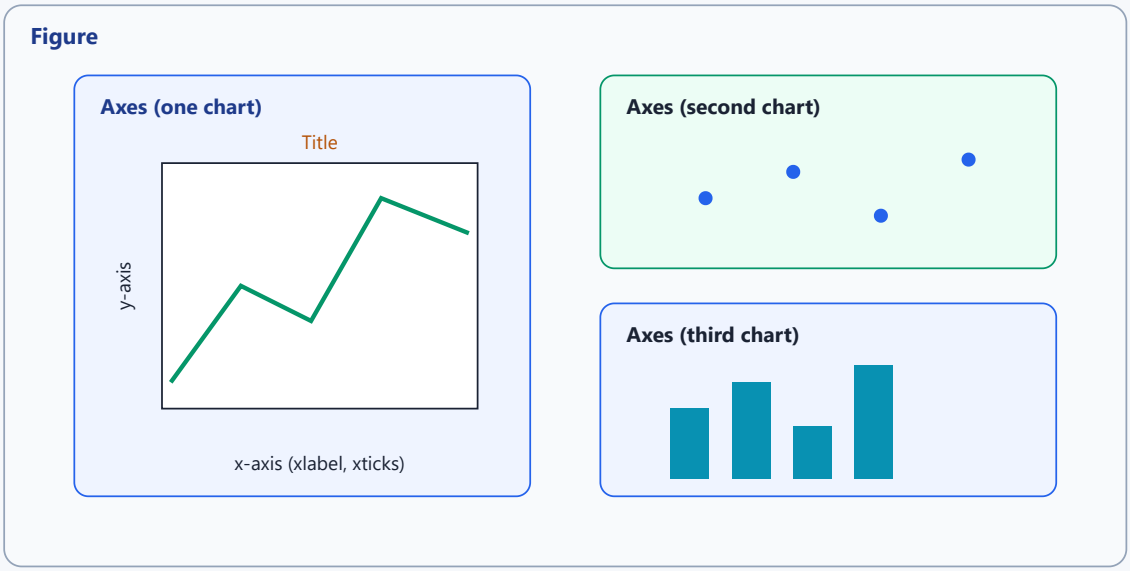


Fig 2 — One Figure contains one or more Axes. Each Axes owns its own data region, two Axis objects (x/y), ticks, labels, title, and legend.

PYTHON

```python
# The explicit, recommended pattern
fig, ax = plt.subplots(figsize=(6, 4))   # one Figure, one Axes
x = np.linspace(0, 2 * np.pi, 200)
ax.plot(x, np.sin(x), label="sin")
ax.plot(x, np.cos(x), label="cos", linestyle="--")

ax.set_title("Trig functions")
ax.set_xlabel("x (radians)")
ax.set_ylabel("f(x)")
ax.legend()
ax.grid(True, alpha=0.3)
fig.savefig("trig.png")   # or plt.show()
```

⚠ **Gotcha**

`plt.plot` and `ax.plot` look identical but `plt` targets whatever Axes was created last. Inside a loop or a helper function this silently draws onto the wrong chart. Always carry an explicit `ax` once you have more than one plot.

4. Core Chart Types in Matplotlib

Line, scatter, bar, histogram

PYTHON

```python
rng = np.random.default_rng(0)

fig, axes = plt.subplots(2, 2, figsize=(10, 7))

# Line — trends over an ordered axis (time, sequence)
t = np.arange(0, 50)
axes[0, 0].plot(t, np.cumsum(rng.normal(size=50)), color="#2563eb")
axes[0, 0].set_title("Line: random walk")

# Scatter — relationship between two numeric variables
x = rng.normal(size=200)
y = 2 * x + rng.normal(size=200)
axes[0, 1].scatter(x, y, s=18, alpha=0.6, color="#059669")
axes[0, 1].set_title("Scatter: y vs x")

# Bar — compare a numeric value across categories
cats = ["A", "B", "C", "D"]
axes[1, 0].bar(cats, [23, 17, 35, 29], color="#0891b2")
axes[1, 0].set_title("Bar: value by category")

# Histogram — distribution of one numeric variable
axes[1, 1].hist(rng.normal(size=1000), bins=30, color="#b45309", edgecolor="white")
axes[1, 1].set_title("Histogram: distribution")

fig.tight_layout()   # prevents overlapping labels
```

✦ **tight_layout vs constrained_layout**

`fig.tight_layout()` adjusts spacing once. For complex grids prefer creating the figure with `plt.subplots( ... , layout="constrained")`, which re-flows automatically as you add titles, legends, and colorbars.

Styling anatomy: colors, markers, linestyles

Parameter	Common values	Example
`color` / `c`	name, hex, RGB tuple, "C0".."C9"	`color="#2563eb"`
`linestyle` / `ls`	`"-"` `"--"` `"-."` `":"`	`ls="--"`
`linewidth` / `lw`	points (float)	`lw=2.5`
`marker`	`"o" "s" "^" "x" "." "D"`	`marker="o"`
`alpha`	0 (transparent) to 1	`alpha=0.5`
`cmap`	`viridis`, `magma`, `coolwarm`	`cmap="viridis"`

5. Subplots and Layout

`plt.subplots(nrows, ncols)` returns the Figure and an array of Axes. Index it to place each chart. For non-uniform grids, reach for `subplot_mosaic` — readable and flexible.

PYTHON

```
# A dashboard: one wide chart on top, two below
fig, axd = plt.subplot_mosaic(
    """
    AA
    BC
    """,
    figsize=(10, 6), layout="constrained",
)
axd["A"].plot(np.cumsum(rng.normal(size=100)))
axd["A"].set_title("Overview")
axd["B"].hist(rng.normal(size=500), bins=25)
axd["C"].scatter(rng.normal(size=100), rng.normal(size=100), s=12)

# Share axes so panels are comparable
fig2, (ax1, ax2) = plt.subplots(1, 2, sharey=True, figsize=(9, 3.5))
```

> **Shared scales**
>
> Use `sharex` / `sharey` when panels show the same quantity — comparing panels with different hidden scales is one of the most common ways a chart misleads.

6. Seaborn: Statistical Plots in One Line

Seaborn's modern API splits into **figure-level** functions (create their own Figure, support faceting via `col` / `row`) and **axes-level** functions (draw into an existing `ax`). Knowing which is which explains most "why can't I combine these?" confusion.



Fig 3 — Seaborn's two tiers. Each figure-level function wraps a family of axes-level kinds via its `kind=` argument.

PYTHON

```python
# Built-in example datasets make Seaborn easy to learn
tips = sns.load_dataset("tips")      # restaurant bills
penguins = sns.load_dataset("penguins")
flights = sns.load_dataset("flights")

# Axes-level: draw into a Matplotlib ax you control
fig, ax = plt.subplots(figsize=(6, 4))
sns.scatterplot(data=tips, x="total_bill", y="tip",
                hue="time", size="size", ax=ax)
ax.set_title("Tip vs bill")

# Figure-level: Seaborn makes the Figure, facet with col=
g = sns.relplot(data=tips, x="total_bill", y="tip",
                col="time", hue="smoker", kind="scatter")
g.set_axis_labels("Bill ($)", "Tip ($)")
```

> ⚠ **Figure-level + ax**
>
> You cannot pass `ax=` to `relplot` / `catplot` / `displot` — they build their own Figure. If you need to place a Seaborn chart inside your own subplot grid, use the *axes-level* twin (`scatterplot`, `boxplot`, ...).

The statistical workhorses

PYTHON

```python
# 1) Heatmap — matrix of values; perfect for a correlation matrix
corr = penguins.select_dtypes("number").corr(numeric_only=True)
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm",
            vmin=-1, vmax=1, center=0, ax=ax)

# 2) Boxplot — distribution + outliers across categories
sns.boxplot(data=tips, x="day", y="total_bill", hue="smoker")

# 3) Pairplot — every pairwise scatter + per-variable histograms
sns.pairplot(penguins, hue="species", corner=True)

# 4) Regplot — scatter with a fitted regression line + CI band
sns.regplot(data=tips, x="total_bill", y="tip",
            scatter_kws={"alpha": 0.4}, line_kws={"color": "#b45309"})
```

✦ **Correlation heatmaps**

Always set a **diverging** colormap (`coolwarm` , `RdBu_r`) with `center=0` for correlations, so positive and negative read as opposite hues and zero is neutral. A sequential map here is misleading.

Distributions: histogram vs KDE vs ECDF

PYTHON

```
fig, axes = plt.subplots(1, 3, figsize=(12, 3.5), layout="constrained")
sns.histplot(data=penguins, x="flipper_length_mm", hue="species",
             kde=True, ax=axes[0])
sns.kdeplot(data=penguins, x="flipper_length_mm", hue="species",
            fill=True, alpha=0.4, ax=axes[1])
sns.ecdfplot(data=penguins, x="flipper_length_mm", hue="species", ax=axes[2])
for a, t in zip(axes, ["hist + kde", "kde", "ecdf"]):
    a.set_title(t)
```

7. Worked Mini-Example: An EDA Panel

A realistic first pass on a dataset: understand distributions, relationships, and group differences on one page. This mixes Seaborn (for the statistics) with Matplotlib (for the layout).

PYTHON

```python
import seaborn as sns, matplotlib.pyplot as plt

sns.set_theme(style="whitegrid")
df = sns.load_dataset("penguins").dropna()

fig, axd = plt.subplot_mosaic(
    """
    AB
    CD
    """,
    figsize=(11, 8), layout="constrained",
)

# A: distribution of a key measurement by group
sns.histplot(df, x="body_mass_g", hue="species", kde=True, ax=axd["A"])
axd["A"].set_title("Body mass distribution")

# B: relationship between two measurements, colored by group
sns.scatterplot(df, x="flipper_length_mm", y="body_mass_g",
                hue="species", style="sex", ax=axd["B"])
axd["B"].set_title("Flipper vs mass")

# C: group comparison with box + raw points overlaid
sns.boxplot(df, x="species", y="bill_length_mm", ax=axd["C"], fliersize=0)
sns.stripplot(df, x="species", y="bill_length_mm", ax=axd["C"],
              color="black", alpha=0.3, size=3)
axd["C"].set_title("Bill length by species")

# D: correlation structure
corr = df.select_dtypes("number").corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm",
            center=0, ax=axd["D"])
axd["D"].set_title("Correlations")

fig.suptitle("Penguins — exploratory overview", fontsize=15)
fig.savefig("eda_penguins.png", dpi=150)
```

✓ **Why this works**

Four complementary views answer four questions — "how is it distributed?", "what relates to what?", "do groups differ?", "which variables move together?" — the standard opening moves of exploratory data analysis.

8. Choosing the Right Chart

Pick the chart from the *question* and the *data types*, not from aesthetics. The table below is the decision shortcut.

Your question	Data	Chart	Seaborn
How does one variable change over an ordered axis?	numeric vs time/sequence	Line	`lineplot`
Do two numeric variables relate?	numeric vs numeric	Scatter (+ regline)	`scatterplot`, `regplot`
How is one variable distributed?	one numeric	Histogram / KDE / ECDF	`histplot`, `kdeplot`
Compare a value across categories?	category vs numeric	Bar	`barplot`, `countplot`
Compare distributions across categories?	category vs numeric	Box / Violin	`boxplot`, `violinplot`
How do many variables correlate?	matrix	Heatmap	`heatmap`
Explore all pairwise relationships?	several numeric	Scatter matrix	`pairplot`
Part-to-whole over few categories?	category + proportion	Stacked/grouped bar (not pie)	`barplot`

Question -> **Data types** -> **Chart family** -> **Polish**

9. Cheat-Sheet Reference

Task	Code
Create figure + axes	`fig, ax = plt.subplots(figsize=(6,4))`
Grid of axes	`fig, axes = plt.subplots(2, 3)`
Set titles/labels	`ax.set(title=…, xlabel=…, ylabel=…)`
Legend	`ax.legend(loc="best")`
Limits / ticks	`ax.set_xlim(0,10); ax.set_xticks([…])`
Log scale	`ax.set_yscale("log")`
Annotate a point	`ax.annotate("peak", xy=(x,y), xytext=(x,y+1), arrowprops={…})`
Save high-res	`fig.savefig("f.png", dpi=300, bbox_inches="tight")`
Close figure (free memory)	`plt.close(fig)`
Seaborn theme	`sns.set_theme(style="whitegrid")`
Seaborn palette	`sns.set_palette("colorblind")`
Facet grid	`sns.relplot(…, col="g", row="h")`
Move legend outside	`sns.move_legend(ax, "upper left", bbox_to_anchor=(1,1))`

Matplotlib vs Seaborn at a glance

Aspect	Matplotlib	Seaborn
Level	Low — explicit artists	High — statistical charts
Input	arrays, anything	tidy DataFrames (best)
Stats (CI, aggregation)	manual	built in
Defaults	plain	attractive themes
Faceting	manual subplots	one-line `col` / `row`
Final control	total	drop to Matplotlib

10. Best Practices

✓ **Do**

Prefer the object-oriented API (`fig, ax`). Label every axis with units. Start bar/count axes at zero. Use colorblind-safe palettes (`sns.set_palette("colorblind")` or the `viridis` family). Keep aspect ratios honest. Export vector (`.svg` / `.pdf`) for print, high-DPI PNG for screens.

✦ **Legibility**

Fewer than ~7 colors; direct-label lines instead of relying on a legend when you can; use `layout="constrained"` to stop labels colliding; set a `context` (`paper` / `notebook` / `talk` / `poster`) to scale fonts for the medium.

⚠ **Avoid**

Pie charts for more than 2-3 slices (angles are hard to compare — use a bar). Dual y-axes that imply false correlation. Rainbow (`jet`) colormaps — they create perceptual banding and fail for colorblind readers.

✕ **Deceptive axes**

Truncating a bar-chart y-axis exaggerates differences and is a classic way to lie with data. Bars encode length, so their baseline **must** be zero. Line charts, which encode position, may use a non-zero baseline when justified.

11. Common Pitfalls

MEMORY LEAKS IN LOOPS

Creating hundreds of figures without `plt.close(fig)` keeps them all in memory. Close each one after saving in batch jobs.

WRONG CURRENT AXES

Mixing `plt.*` and `ax.*` draws onto the last-created Axes. Commit to the explicit API with multiple plots.

OVERLAPPING LABELS

Default layout can clip titles/ticks. Use `layout="constrained"` or `fig.tight_layout()`; rotate dense tick labels.

UNTIDY DATA FOR SEABORN

Seaborn wants long/tidy format (one row per observation). Reshape wide tables with `df.melt()` before plotting.

BLURRY EXPORTS

Default DPI is low. Pass `dpi=150–300` and `bbox_inches="tight"`, or save vector formats.

SEQUENTIAL MAP FOR DIVERGING DATA

Correlations and signed deltas need a diverging map centered at 0, not `viridis`.

Show: reshape a wide table to tidy for Seaborn

12. Interview Questions & Answers

What is the difference between a Figure and an Axes in Matplotlib?

The Figure is the top-level container (the whole canvas); an Axes is a single plot drawn within it, with its own data region, x/y Axis objects, ticks, title, and legend. One Figure can hold many Axes. Note the terminology trap: an *Axes* is a whole subplot, while an *Axis* (singular) is one of its two coordinate axes.

When should you use the pyplot interface vs the object-oriented interface?

Use pyplot (`plt.plot`) for quick, single-chart exploration. Use the OO interface (`fig, ax = plt.subplots()` then `ax.method()`) for anything with multiple subplots, inside functions, or in production code — it is explicit about which Axes you target and avoids hidden global state.

How does Seaborn relate to Matplotlib?

Seaborn is a higher-level library built on top of Matplotlib. It computes statistics (aggregations, confidence intervals, KDEs) and applies attractive themes, but it renders by creating Matplotlib Figure/Axes objects. Axes-level Seaborn functions accept `ax=` , so you can draw a Seaborn chart and then fine-tune it with raw Matplotlib calls.

What is the difference between figure-level and axes-level Seaborn functions?

Axes-level functions (`scatterplot` , `boxplot` , `heatmap`) draw into a single provided Axes and return it. Figure-level functions (`relplot` , `catplot` , `displot` , `pairplot`) manage their own Figure, return a `FacetGrid` , and support faceting via `col` / `row` — but you cannot pass them an existing `ax` .

When would you choose a boxplot over a histogram?

Use a histogram/KDE to see the full shape of one distribution (modality, skew). Use a boxplot to compactly compare the summary (median, quartiles, outliers) of a numeric variable *across several categories* side by side. For small samples, overlay the raw points (strip/swarm) because a box can hide how few observations there are.

Why must a bar chart's y-axis start at zero but a line chart's need not?

A bar encodes value through *length*, so a non-zero baseline distorts the ratios the length is supposed to communicate. A line encodes value through *position*, so zooming the axis to the data range is legitimate and often necessary to see trends — provided the axis is clearly labelled.

What colormap would you use for a correlation matrix and why?

A diverging map such as `coolwarm` or `RdBu_r` with `center=0` and `vmin=-1, vmax=1` . Correlations are signed and symmetric around zero, so you want two opposing hues meeting at a neutral middle. A sequential map (like `viridis`) would misrepresent strong negative and strong positive values as similarly "extreme" only by intensity, losing the sign.

What does "tidy" or "long-form" data mean for Seaborn, and why does it matter?

> Tidy data has one row per observation and one column per variable, so categorical variables live in their own columns to be mapped onto `x`, `hue`, `col`, etc. Seaborn's declarative API relies on this: you name columns and it does the grouping. Wide data (a column per series) must be reshaped with `df.melt()` first.

> **How do you prevent a memory blow-up when generating many figures in a batch job?**
>
> Each `plt.figure` / `subplots` stays registered in pyplot's state machine until closed. After saving, call `plt.close(fig)` (or `plt.close("all")`) so the figure and its artists are garbage-collected. On a headless server also set the `Agg` backend so no GUI resources are allocated.

> **How do you place a legend outside the plotting area?**
>
> With Matplotlib: `ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1))` and use `constrained` layout so it isn't clipped. With Seaborn axes-level plots, `sns.move_legend(ax, "upper left", bbox_to_anchor=(1, 1))` is the clean one-liner; figure-level grids expose `g.legend`.

> **What is the difference between `regplot` and `lmplot`?**
>
> Both fit and draw a regression line with a confidence band. `regplot` is axes-level — it draws into one Axes you control. `lmplot` is figure-level — it wraps `regplot` on a FacetGrid, so it can split the regression across subplots with `col` / `row` / `hue`. Choose `regplot` to compose into a custom layout, `lmplot` for quick faceted regressions.

> **How would you visualise the relationship between every pair of numeric columns quickly?**
>
> `sns.pairplot(df, hue="group", corner=True)` draws a matrix of pairwise scatterplots with per-variable distributions on the diagonal; `hue` colors by a category and `corner=True` drops the redundant upper triangle. For a numeric summary of the same thing, compute `df.corr()` and show it as a `heatmap`.

13. Further Learning

- Matplotlib official *Users Guide* and the *Anatomy of a Figure* and *cheatsheets* pages — authoritative and current.
- Seaborn's *official tutorial* (especially "Overview of seaborn plotting functions" and "Data structures accepted by seaborn").
- The newer **seaborn.objects** interface (`import seaborn.objects as so`) — a composable grammar-of-graphics API worth exploring once the basics click.
- Claus Wilke, *Fundamentals of Data Visualization* (free online) — chart choice and perception, library-agnostic.

- Practice: load `sns.get_dataset_names()` datasets and rebuild each chart type from memory, then recreate a chart you admire from a paper or dashboard.
- Next steps in the stack: interactive plots with **Plotly**/**Altair**, and large-data rendering with **datashader**.